

LAPORAN PRAKTIKUM INTERNET OF THINGS II

Komunikasi Asinkron dan WebSockets



Oleh:

NAMA : Albertus Satriyo Nugroho
NPM : 25782002
KELAS : 3A
MATKUL : Internet of Things (IOT)

Dosen:

**Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T. Agus
Ambarwari, S.Pd., M.Kom.**

**TEKNOLOGI REKAYASA INTERNET JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG**

Latihan

1. jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

JAWAB:

karena hanya membutuhkan satu kali *handshake* di awal. Setelah terhubung, data dikirim langsung lewat jalur tetap dengan *overhead header* super kecil (beberapa byte saja), tanpa perlu mengirim ulang data HTML/CSS yang sama sedangkan pada Meta Refresh, browser dipaksa meminta ulang seluruh halaman web dari awal lengkap dengan *header* HTTP yang besar setiap beberapa detik.

2. Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

JAWAB:

- Tentunya NodeMCU akan gagal merespon sinyal heartbeat sehingga browser menganggap koneksi mati dan memutuskan Websocet.
- lalu menyebabkan Data Amburadul Perintah dari browser tertahan di *buffer* respons jadi sangat lambat dan paket data rawan hilang.
- Crash/Reboot Fitur *Watchdog Timer* (WDT) menganggap sistem *hang* lalu memaksa NodeMCU *restart* mendadak.

3. Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML () apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

JAWAB:

Yang saya ubah, status – Menampilkan status koneksi WebSocket secara *real-time*, state – Mengubah teks kondisi alat, "ON" atau "OFF" secara *real-time*, btnText – Mengubah label di dalam tombol, dari "Nyalakan" jadi "Matikan" sesuai kondisi saklar saat itu.

4. Pengembangan Telemetry Sensor – Kelembapan

JAWAB:

1. Sisi Back-End (C++)

Bagian kode di NodeMCU untuk membaca sensor DHT11 non-blocking dan menyisipkan data kelembapan ke format JSON:

```
// 1. Variabel penampung data kelembapan & suhu
String currentHum = "--";
String currentTemp = "--";
String ledState = "OFF";

unsigned long previousMillis = 0;
const long interval = 2000; // Pembacaan tiap 2 detik
```

```
// 2. Fungsi broadcast paket JSON terintegrasi
void notifyClients() {
    String jsonPayload = "{\"led\": \"" + ledState + "\", \"suhu\": \"" + currentTemp + "\", \"hum\": \"" +
currentHum + "\"}";
    ws.textAll(jsonPayload);
}

// 3. Pembacaan sensor di void loop()
void loop() {
    ws.cleanupClients();

    unsigned long currentMillis = millis();
    if (currentMillis - previousMillis >= interval) {
        previousMillis = currentMillis;

        float h = dht.readHumidity();
        float t = dht.readTemperature();

        if (!isnan(h) && !isnan(t)) {
            currentHum = String(h, 1);
            currentTemp = String(t, 1);
            notifyClients(); // Kirim data terbaru via WebSocket
        }
    }
}
```

2. Sisi Front-End (HTML & JavaScript)

A. Elemen HTML (Penambah Kartu Kelembapan)

```
<div class="card">
  <h2>Kelembapan: <span id="humValue">--</span> %</h2>
</div>
```

B. JavaScript (Fungsi onMessage)

```
function onMessage(event) {
    let data = JSON.parse(event.data);

    // Ekstrak data kelembapan dari JSON & update UI
    if (data.hum !== undefined) {
        document.getElementById('humValue').innerText = data.hum;
    }
    if (data.suhu !== undefined) {
        document.getElementById('tempValue').innerText = data.suhu;
    }
    if (data.led !== undefined) {
        document.getElementById('state').innerText = data.led;
        document.getElementById('btnText').innerText = (data.led === "ON") ? "Matikan" : "Nyalakan";
    }
}
```